Lab 3 — Deploy it, then watch it

13 October · 3h15 · one Group at a time · after Lecture 2 Serves: C2, C3

You start with

- Your fork's default branch runs the Architecture Check and your Lab 2 `test` job on GitHub, and its last run is green. Behind? Take the CI Fallback Branch, `course-2026-fallback-ci`: push it to your fork and make it your default, as in step 3 of the "Required checks — a short guide" (document available in the Resources section). Then add your Lab 2 `test` job, running `pytest`, to its workflow.
- An Azure for Students account, the Azure CLI (the `az` command) signed in to it, and a Mistral API key of your own. `az account show` names your Azure for Students subscription. Behind? Open them now, as "Your Azure and Mistral accounts — before Lab 3" (document available in the Resources section) says, before step 1 needs them.
- The GitHub CLI signed in to your account: `gh auth status` names you. Behind? Run `gh auth login` before step 1.
- Your Design Document, an issue on your Project's upstream. Behind? Open it before step 7, which hangs your Error Ledger under it.
- Your production dev sign-in key and your URL, in the Instructor's Teams message. Behind? Ask the Instructor at the start of step 1.

You'll leave with

- A pipeline that deploys each green commit to Fork staging, then promotes it to Fork production.
- Your production signals in your own PostHog project, proven with your own exception.
- Fork production calling its LLM through the URL you are given, and named in your fork's `production.json`.
- Your Error Ledger, its first entry copied from your saved query.

New words

- **Deploy on green**: your fork is deployed only from a pushed commit whose CI run passed.
- **Promotion**: Fork production only receives the image Fork staging already received. Your Promotion is the workflow you start by hand to do it.
- **Production signals**: what your fork reports to its own PostHog project: its errors and its logs.
- **GitHub Environment**: a named set of variables and secrets in your fork's settings, given only to a workflow job that runs in it.

Preliminary Step: update Matt Pocock's skills to v1.3 (5 min, before step 1)

Done when

Matt Pocock's skills, if your coding agent uses them, show v1.3 or later, and no file on the branch you work from still names `CONTEXT.md`.

1. Create your Azure resources (0:00–0:20)

You start with

- Your Azure for Students account, with the Azure CLI signed in to it.

Why this step matters

A pipeline needs somewhere to deploy, and an identity to deploy as. This step creates both, for a pipeline that deploys each green commit to Fork staging, then promotes it to Fork production.

Do

1. Give your coding agent this Lab page and its Agent Brief.
2. Create your Azure resources yourself, one command at a time. They spend your credit and grant access, so your agent only reads them. On Windows, type them in PowerShell, not Git Bash. Replace each `<…>`. `<you>` is your GitHub username in lowercase, and `<project>` is `oceens` or `mathutrice`. Each app's name is its address, so it must be unique.

```
# Your region: the first region the command prints that is on this list.
# francecentral, westeurope, northeurope, germanywestcentral, belgiumcentral,
# switzerlandnorth, italynorth, austriaeast, spaincentral, swedencentral,
# polandcentral, norwayeast, denmarkeast, uksouth, ukwest
az policy assignment list --query "[?displayName=='Allowed resource deployment
regions'].parameters.listOfAllowedLocations.value[]" -o tsv
# One resource group holds everything below, so you see it, and delete it, as one
az group create --name <you>-forks --location <region>
# The plan is the machine: Linux, Basic B1, about $21–23 of your $100 over the Course,
billed by the hour, used or not
az appservice plan create --name <you>-forks --resource-group <you>-forks --location
<region> --sku B1 --is-linux
```

```
# Two apps on that one machine. The image is a placeholder your first deploy replaces
az webapp create --name <you>-<project>-staging --resource-group <you>-forks --plan
<you>-forks --container-image-name mcr.microsoft.com/appsvc/staticsite:latest --https-
only true
az webapp create --name <you>-<project>-production --resource-group <you>-forks --plan
<you>-forks --container-image-name mcr.microsoft.com/appsvc/staticsite:latest --https-
only true
```

```
# Always on: otherwise an idle app stops, and the next request waits for it to start
az webapp config set --name <you>-<project>-staging --resource-group <you>-forks --
always-on true
az webapp config set --name <you>-<project>-production --resource-group <you>-forks --
always-on true
# Keep what each app's container prints, so a start that fails says why in its log
az webapp log config --name <you>-<project>-staging --resource-group <you>-forks --
docker-container-logging filesystem
az webapp log config --name <you>-<project>-production --resource-group <you>-forks --
docker-container-logging filesystem
```

```
# The identity your pipeline signs in as. It has no password to leak.
# Keep the clientId and principalId it prints
az identity create --name github-deploy --resource-group <you>-forks
# How GitHub names your fork to Azure, with its owner's and its own number.
# <your fork> is owner/name.
# Keep what it prints, such as repo:octocat@583231/OceENS@1296269
gh api repos/<your fork>/actions/oidc/customization/sub --jq .sub_claim_prefix
```

```
# The identity trusts GitHub for your fork's two environments, and nothing else.
# <your subject> is what gh api printed, as it printed it
az identity federated-credential create --name staging --identity-name github-deploy --
resource-group <you>-forks --issuer https://token.actions.githubusercontent.com --
subject <your subject>:environment:staging --audiences api://AzureADTokenExchange
az identity federated-credential create --name production --identity-name github-deploy
--resource-group <you>-forks --issuer https://token.actions.githubusercontent.com --
subject <your subject>:environment:production --audiences api://AzureADTokenExchange
# It may deploy to the apps of this resource group alone.
# The scope is the id az group create printed
az role assignment create --assignee-object-id <principalId> --assignee-principal-type
ServicePrincipal --role "Website Contributor" --scope <the resource group's id>
```

3. Create your fork's two GitHub Environments, `staging` and `production`, and give each the five values your pipeline signs in and deploys with. They are variables, not secrets: none opens anything on its own. Name your fork, `owner/name`, in every command: otherwise `gh` may write to the Project's upstream.

```
# The two environments your pipeline deploys to, one for each app
gh api -X PUT repos/<your fork>/environments/staging
gh api -X PUT repos/<your fork>/environments/production
```

```
# The identity's clientId, from az identity create
gh variable set AZURE_CLIENT_ID --env staging --body <clientId> --repo <your fork>
gh variable set AZURE_CLIENT_ID --env production --body <clientId> --repo <your fork>
# Your tenant: az account show --query tenantId -o tsv
gh variable set AZURE_TENANT_ID --env staging --body <tenantId> --repo <your fork>
gh variable set AZURE_TENANT_ID --env production --body <tenantId> --repo <your fork>
# Your subscription: az account show --query id -o tsv
gh variable set AZURE_SUBSCRIPTION_ID --env staging --body <subscription id> --repo
<your fork>
gh variable set AZURE_SUBSCRIPTION_ID --env production --body <subscription id> --repo
<your fork>
```

```
# Your resource group
gh variable set AZURE_RESOURCE_GROUP --env staging --body <you>-forks --repo <your
fork>
gh variable set AZURE_RESOURCE_GROUP --env production --body <you>-forks --repo <your
fork>
```

```
# Each environment's own app
gh variable set AZURE_WEBAPP_NAME --env staging --body <you>-<project>-staging --repo
<your fork>
gh variable set AZURE_WEBAPP_NAME --env production --body <you>-<project>-production --
repo <your fork>
```

See the Worked Example

The other Project's pipeline signs in with these five variables, in its *Log in to Azure* step. **OcéENS:** MATHutrice's EPF-MDE/MATHutrice#100. **MATHutrice:** OcéENS's EPF-MDE/OceENS#169.

Done when

`az webapp list --resource-group <you>-forks -o table` lists your two apps, each answers at its address, and each GitHub Environment holds its five Azure variables.

Stuck?

- **The region command prints no region of the list?** Create nothing. Show its output to the Instructor.
- **A command failed?** Read its error, fix the value, and run that command again. No Fallback Branch creates your Azure resources: they are yours.
- **Your first deploy fails at *Log in to Azure* with `AADSTS700213`?** Your credential's subject is not the one GitHub sent. Copy the subject from the error: it is in quotes after `assertion subject`. Run that environment's `az identity federated-credential create` command again, with that subject after `--subject`. Then run the deploy again.

You should be able to

Say which resource spends your credit, and what someone who changed your fork's workflow could do as your identity.

2. Deploy each green commit to Fork staging (0:20–1:00)

You start with

- Your fork's default branch runs the Architecture Check and your Lab 2 `test` job on GitHub, and its last run is green.
- Your two apps, the identity your pipeline signs in as, and your two GitHub Environments with their Azure variables, from step 1.
- Your Mistral key, and the production dev sign-in key from the Instructor's Teams message.

Why this step matters

What you watch means nothing until you know which commit is running, so the pipeline comes first. This step builds the first half of a pipeline that deploys each green commit to Fork staging, then promotes it to Fork production.

Do

1. Give each GitHub Environment its secrets. Your deploy writes them to its app, so each app's settings live beside the pipeline that uses them. Each environment gets its own secrets, so staging and production never share a session. Never commit them, and never use `.env.example`'s values. Your laptop, staging and production spend one Mistral quota: a run on your laptop can cause a `429` in Fork production.

   Fork staging and Fork production run the same image, and both sign in through the development sign-in. Its key decides who gets in: without one, anyone signs in as anyone. Staging's key is yours, and only you use it. Production's is the key the Instructor gives you, because the simulated users sign in to Fork production with it. Never put one environment's key in the other.

   ```
   # Make a random value. Run it once for each value below
   uv run python -c "import secrets; print(secrets.token_hex(24))"
   # Staging's key: a value you made. Keep it with your passwords.
   # gh asks for the value: paste it
   gh secret set DEV_LOGIN_KEY --env staging --repo <your fork>
   # Production's key: paste the key the Instructor gave you
   gh secret set DEV_LOGIN_KEY --env production --repo <your fork>
   # Each environment's session secret: a new value you made for each.
   # OcéENS: SECRET_KEY. MATHutrice: SESSION_SECRET
   gh secret set <SECRET_KEY or SESSION_SECRET> --env staging --repo <your fork>
   gh secret set <SECRET_KEY or SESSION_SECRET> --env production --repo <your fork>
   # Your Mistral key, the same in both. OcéENS: MISTRAL_API_KEY. MATHutrice: LLM_API_KEY
   gh secret set <MISTRAL_API_KEY or LLM_API_KEY> --env staging --repo <your fork>
   gh secret set <MISTRAL_API_KEY or LLM_API_KEY> --env production --repo <your fork>
   ```

   Every other setting is not a secret, and is the same in both environments. Your agent writes it in the workflow, which passes it to each app at each deploy. You store nothing more.

2. List every process the old launcher started. Check that your deploy starts each one.

3. Deploy each green commit of your default branch to Fork staging, and wait until the new version answers. Say in your README how your fork is deployed, so that its documentation stays true. **OcéENS:** once deployed, test the LLM row in `/backend/providers`.

4. After your first deploy pushes your image, open its package's *Package settings* on GitHub. If it is Private, make it Public: Azure pulls it without a password.

See the Worked Example

The other Project's pipeline, and its green run. **OcéENS:** MATHutrice's EPF-MDE/MATHutrice#100, and its green run. **MATHutrice:** OcéENS's EPF-MDE/OceENS#169, and its green run.

Done when

Fork staging runs your last pushed commit, deployed by the run that passed on it, with every process the old launcher started.

Stuck?

- **The deploy times out, and your app's log says `ImagePullFailure`, forbidden or unauthorized?** Check your package's visibility, as in *Do*, then run the deploy again.
- **The deploy fails at *Log in to Azure* with `AADSTS700213`?** Recreate the credential with the subject in the error, as step 1's *Stuck?* says.
- **Fork not answering on the host at 0:45?** Take the Ready-to-Deploy Fallback Branch, `course-2026-fallback-ready-to-deploy`: push it to your fork, then make it your default branch in your fork's *Settings* → *General* → *Default branch*. Merge your Lab 2 work into it in one pull request. It runs on the host, but it has no PostHog wiring, and it is not deployed.
- **No pipeline?** There is no Deploy Fallback Branch. Your pipeline is yours to write, on your own branch or on the Ready-to-Deploy Fallback Branch, because C2 grades it. The Worked Example is your help.
- A Fallback Branch costs the experience, not the grade. You still write your own pipeline.

You should be able to

Reject any deploy in your agent's workflow that does not wait for the check. It is the first requirement in the table of "Lecture 1 — Nothing is designed until a number asks for it" (document available in the Lectures section).

3. Promote staging's image to Fork production (1:00–1:20)

You start with

- Fork staging running your last pushed commit, deployed by the run that passed on it, with every process the old launcher started.

Why this step matters

Fork production must only run what staging already ran, so nothing reaches it untested. This step finishes the pipeline that deploys each green commit to Fork staging, then promotes it to Fork production.

Do

1. Add a Promotion you start by hand: it deploys the image of staging's last green deploy to Fork production, and builds nothing.
2. Run it once.

See the Worked Example

A Promotion skipping a failed staging deploy. **OcéENS:** MATHutrice's failed deploy and Promotion. **MATHutrice:** OcéENS's failed deploy and Promotion.

Done when

Fork production runs staging's image, deployed by a Promotion run.

Stuck?

No Fallback Branch carries a Promotion either, because C2 grades it with your pipeline. Read how the Worked Example's Promotion skips the failed deploy. Not done at 1:20? Go on to step 4, and finish your Promotion before step 5 needs it.

You should be able to

Say what your Promotion does when staging's last deploy failed.

4. Wire your production signals into PostHog (1:20–2:05)

You start with

- A deploy that starts every process your service runs, from step 2.
- Your Lab 2 tests and your Project's smoke test, `docs/smoke-test.md`, passing on your default branch.

Why this step matters

An error that doesn't name who it reached can't be ranked. This step wires your production signals into your own PostHog project, with each error naming its user.

Do

1. Open a free PostHog account in the EU region, with one project. Invite the Instructor, `xavier.cazalot@gmail.com`, to its organization. Change no other setting. PostHog is the Course's default: you may wire another tool instead, but the room gives no help with it.

2. Give each environment its PostHog settings, so that it reports to your project under its own name. All three, or none: with none, PostHog stays off.

```
# Secret: your project's token, starting with phc_
POSTHOG_PROJECT_TOKEN=phc_<your token>
# PostHog's EU region
POSTHOG_HOST=https://eu.i.posthog.com
# What each error and log line is tagged with: staging or production
POSTHOG_ENVIRONMENT=staging
```

3. Have your agent wire Error tracking, identity and Logs, in every process your service runs. It pins `posthog` at `7.60.0` and OpenTelemetry at `1.44.0`: the versions the Worked Example was proven with.

4. Find each place your code catches an exception and answers for itself, and show it to your agent. **An exception your code catches is still an error: it is reported before the user gets an answer.** Otherwise your Fork production looks healthier than it is.

5. **Every exception carries the signed-in user's email.** Otherwise every error reads "1 user". **OcéENS:** the summaries daemon's errors carry the identity `oceens-summaries` and the survey they reached. Count them in surveys, not users.

See the Worked Example

The other Project's wiring. **OcéENS:** MATHutrice's `4510114`, `b8f9a13`, `578e407`. **MATHutrice:** OcéENS's `1c673c9`, `e453266`, `6d36ae5`.

Done when

With the three settings unset, your tests and the smoke test pass as before.

Stuck?

Wiring not done by 2:05? Go on to step 5 with what you have. If your exception does not arrive there, step 5's Fallback Branch gives you the wiring. It costs the experience, not the grade: finding each exception your code catches stays yours.

You should be able to

Point at each place your code catches an exception, and say whether it is reported.

5. Prove the wiring with your own exception on staging (2:05–2:25)

You start with

- Your wiring, from step 4, or as far as you got.
- A pipeline that deploys to Fork staging and promotes to Fork production.

Why this step matters

Wiring you haven't seen catch your own error proves nothing. This step proves your production signals with your own exception, before Fork production gets them.

Do

1. Add a temporary route that raises an exception, and deploy it to staging. Sign in and call it. A `404` means the old version still serves: call again a minute later.

2. Find your exception in Error tracking: your email, `environment` `staging`. Find a line your service wrote in Logs.

3. Remove the route, and deploy staging again.

4. In PostHog's SQL editor, run your Project's query below with `'staging'` in place of `'production'`. Your exception appears. Put `'production'` back, and save it as `Production errors by issue, with users reached`, on a new dashboard, `Fork production`.

   OcéENS:

   ```
   SELECT properties.$exception_issue_id AS issue,
       any(properties.$exception_types) AS types,
       count() AS exceptions, count(DISTINCT distinct_id) AS users,
       count(DISTINCT properties.survey_id) AS surveys,
       min(timestamp) AS first_seen, max(timestamp) AS last_seen
   FROM events
   WHERE event = '$exception' AND properties.environment = 'production'
   GROUP BY issue ORDER BY exceptions DESC
   ```

   MATHutrice:

   ```
   SELECT properties.$exception_issue_id AS issue,
       any(properties.$exception_types) AS types,
       count() AS exceptions, count(DISTINCT distinct_id) AS users,
       min(timestamp) AS first_seen, max(timestamp) AS last_seen
   FROM events
   WHERE event = '$exception' AND properties.environment = 'production'
   GROUP BY issue ORDER BY exceptions DESC
   ```

5. Promote.

See the Worked Example

None for the proof, which is yours. The Promotion of the wiring: **OcéENS:** EPF-MDE/MATHutrice#101, on green, promoted. **MATHutrice:** EPF-MDE/OceENS#170, on green, promoted.

Done when

Your exception arrived with your email, the route is gone, the query is saved, and your wiring is promoted.

Stuck?

No exception of yours by 2:15? Take the PostHog Fallback Branch, `course-2026-fallback-posthog`, as you would the Ready-to-Deploy one in step 2. Keep its wiring. Set your settings, add your own route, and deploy again. It costs the experience, not the grade: the proof stays yours.

You should be able to

Decide honestly whether the exception you saw is yours.

6. Point Fork production at the URL you're given, and find your own log line (2:25–2:45)

You start with

- Your wiring, proven on staging and promoted to Fork production.
- The URL you are given, from the Instructor's Teams message.

Why this step matters

Fork production is the environment watched from now on, so it calls its LLM through the URL you are given, and your fork names its address. This step leaves Fork production calling its LLM through that URL, and named in your fork's `production.json`.

Do

1. Point Fork production's LLM calls at the URL you are given, ending in `/oceens/<token>/v1` or `/mathutrice/<token>/v1`. It relays your Mistral key and never stores it. Staging, like your laptop, calls Mistral directly, so only production's GitHub Environment changes.

   OcéENS: then test the row in `/backend/providers` again.

   ```
   # The URL you are given, in place of step 2's Mistral URL
   DEFAULT_PROVIDER_BASE_URL=<the URL you are given>
   ```

   MATHutrice: staging keeps step 2's, Mistral's own URL.

   ```
   # The URL you are given, in place of Mistral's own
   LLM_BASE_URL=<the URL you are given>
   ```

2. Commit `production.json` at the root of your default branch, with Fork production's URL only: `{ "url": "https://<app>.azurewebsites.net" }`.

3. Sign in to Fork production, and do once an action your code logs. Find its line in Logs, tagged `production`.

See the Worked Example

None: the line is your own, in your PostHog project's Logs, tagged `production`.

Done when

A line your own action wrote is in Logs, tagged `production`, at the time you did it. It is not a line your service writes on its own, when it starts or every few minutes.

Stuck?

No line in Logs? Logs are sent in batches: wait a minute. Then check that your action goes through a `logging` call in your code.

You should be able to

Point at the code that wrote it, and say why a startup line proves nothing.

7. Open your Error Ledger (2:45–3:00)

You start with

- Your saved query, `Production errors by issue, with users reached`, from step 5.
- Your Design Document, an issue on your Project's upstream.

Why this step matters

Watching starts with a place to write down what you see. This step opens your Error Ledger, its first entry copied from your saved query.

Do

1. Open your Error Ledger as a sub-issue of your Design Document. Name the PostHog project it reads.
2. Link your saved query, and copy its rows with every column. No row? Write that, with the date.
3. From now on, record in it each error of Fork production. An error's rate is its count of exceptions divided by the time from `first_seen` to `last_seen`.

See the Worked Example

The other Project's Error Ledger, and the deployment record it reads. **OcéENS:** EPF-MDE/MATHutrice#91 and its deployment record. **MATHutrice:** EPF-MDE/OceENS#161 and its deployment record.

Done when

Your Error Ledger is open, its first entry your query's rows or a dated "no row".

Stuck?

Read the Glossary, then this step again. You built everything this Ledger reads from, and no Fallback Branch writes it for you. Not done at 3:00? Open it after the Lab: it is your own writing, and nobody else's.

You should be able to

Say which row you would look at first, by rate and users reached.

Checklist

- [ ] Matt Pocock's skills, if your coding agent uses them, show v1.3 or later, and no file on the branch you work from still names `CONTEXT.md`.

- [ ] `az webapp list --resource-group <you>-forks -o table` lists your two apps, each answers at its address, and each GitHub Environment holds its five Azure variables.
- [ ] Fork staging runs your last pushed commit, deployed by the run that passed on it, with every process the old launcher started.
- [ ] Fork production runs staging's image, deployed by a Promotion run.
- [ ] With the three settings unset, your tests and the smoke test pass as before.
- [ ] Your exception arrived with your email, the route is gone, the query is saved, and your wiring is promoted.
- [ ] A line your own action wrote is in Logs, tagged `production`, at the time you did it. It is not a line your service writes on its own, when it starts or every few minutes.
- [ ] Your Error Ledger is open, its first entry your query's rows or a dated "no row".